

## Practical - 8

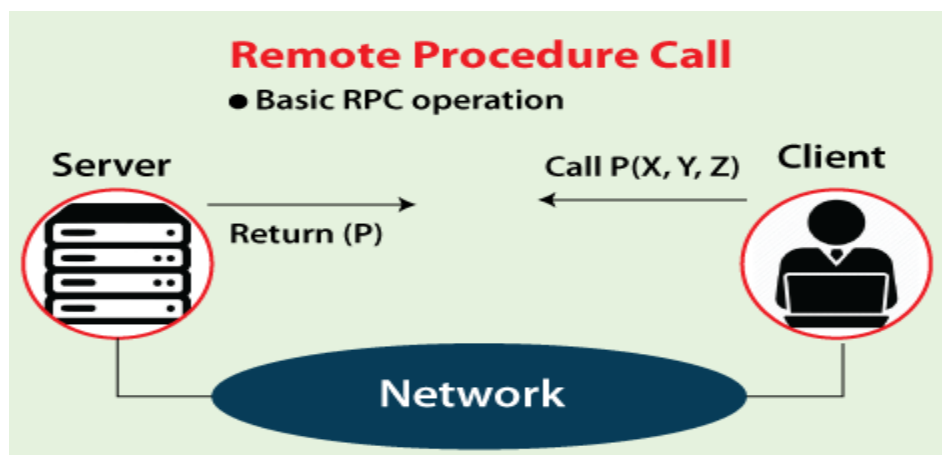
|                                                                                                                                                            |                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| <b>Government College of Engineering, Jalgaon</b><br><b>(An Autonomous Institute of Govt. of Maharashtra)</b><br><b>Department of Computer Engineering</b> |                            |
| <b>Name:</b>                                                                                                                                               |                            |
| <b>PRN:</b>                                                                                                                                                | <b>Batch:</b>              |
| <b>Class:</b>                                                                                                                                              | <b>Semester:</b>           |
| <b>Course:</b>                                                                                                                                             | <b>Subject Teacher:</b>    |
| <b>Date of Performance:</b>                                                                                                                                | <b>Date of Completion:</b> |

**Aim :** Implement RPC mechanism for a file transfer across a network.

### Theory :

#### What is RPC in Operating System?

Remote Procedure Call or RPC is a powerful technique for constructing distributed, client-server-based applications. It is also known as a function call or a subroutine call. A remote procedure call is when a computer program causes a procedure to execute in a different address space, coded as a local procedure call, without the programmer explicitly stating the details for the remote interaction. The programmer writes essentially the same code whether the subroutine is local to the executing program or remote. This is a form of client-server interaction implemented via a request-response message-passing system.



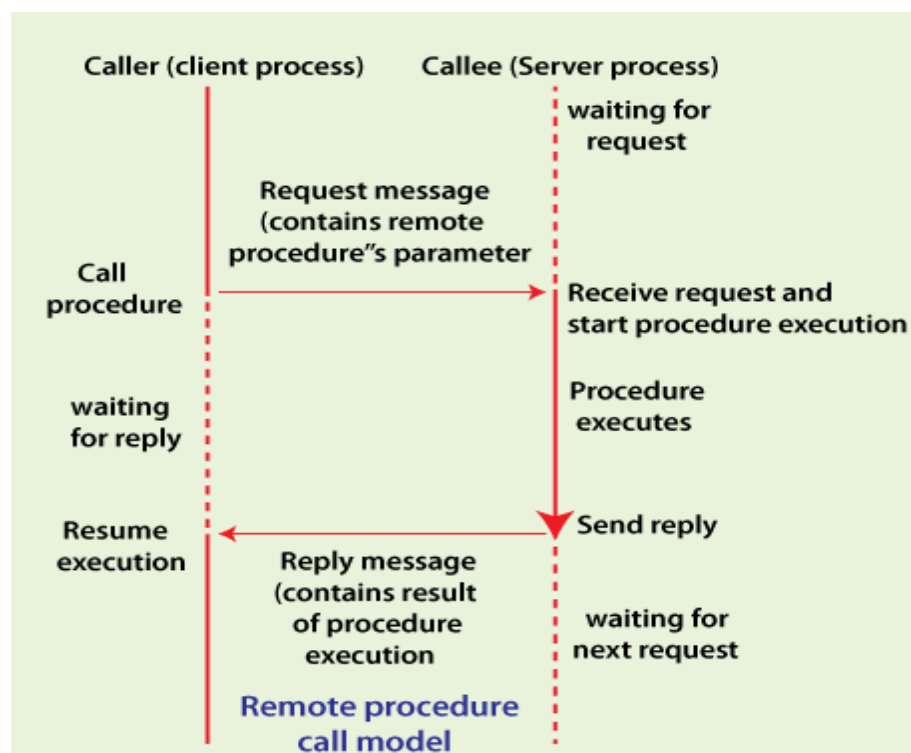
The RPC model implies *location transparency* that calling procedures are largely the same, whether local or remote. Usually, they are not identical, so that local calls can be distinguished

from remote calls. Remote calls are usually orders of magnitude slower and less reliable than local calls, so distinguishing them is important.

RPCs are a form of inter-process communication (IPC), in that different processes have different address spaces. They have distinct virtual address spaces on the same host machine, even though the physical address space is the same. While if they are on different hosts, the physical address space is different.

### How to Make a Remote Procedure Call

The calling environment is suspended, procedure parameters are transferred across the network to the environment where the procedure is to execute, and the procedure is executed there.



When the procedure finishes and produces its results, it is transferred back to the calling environment, where execution resumes as if returning from a regular procedure call.

### Types of RPC

There are three types of remote procedure call (RPC) in an operating system, such as:

#### 1. Callback RPC

This type of RPC enables a P2P paradigm between participating processes. It helps a process to be both client and server services. Callback RPC has the following functions, such as:

- Remotely processed interactive application problems.
- Offers server with clients handle.

- Callback makes the client process wait.
- Manage callback deadlocks.
- It facilitates a peer-to-Peer paradigm among participating processes.

## **2. Broadcast RPC**

Broadcast RPC is a client's request broadcast on the network, processed by all servers with the method for processing that request. Broadcast RPC has the following functions, such as:

- Allows you to specify that the client's request message has to be broadcast.
- You can declare broadcast ports.
- It helps to reduce the load on the physical network.

## **3. Batch-mode RPC**

Batch-mode RPC helps to queue, separate RPC requests, in a transmission buffer, on the client-side, and then send them on a network in one batch to the server. Batch-mode RPC has the following functions, such as:

- It minimizes the overhead involved in sending a request by sending them over the network in one batch to the server.
- This type of RPC protocol is only efficient for an application that needs lower call rates.
- It needs a reliable transmission protocol.

## **Features of RPC**

In an operating system, remote procedure call (RPC) has the following features, such as:

- RPC hides the complexity of the message passing process from the user.
- RPC only uses specific layers of the OSI model like the transport layer.
- Clients can communicate with the server by using higher-level languages.
- RPC works well with both local environments and remote environments.
- The program of RPC is written in simple code and is easily understood by the programmer.
- The operating system can handle processes and threads involved in RPC easily.
- The operating system hides the abstractions of RPC from the user.

## **How RPC works?**

When a remote procedure call is invoked, the calling environment is suspended, the procedure parameters are transferred across the network to the environment where the procedure is to execute, and the procedure is then executed in that environment.

When the procedure finishes, the results are transferred back to the calling environment, where execution resumes as if returning from a regular procedure call.

A remote procedure call (RPC) works in the following steps in an operating system:

**Step 1:** The client, client stub, and RPC run time execute on the client machine.

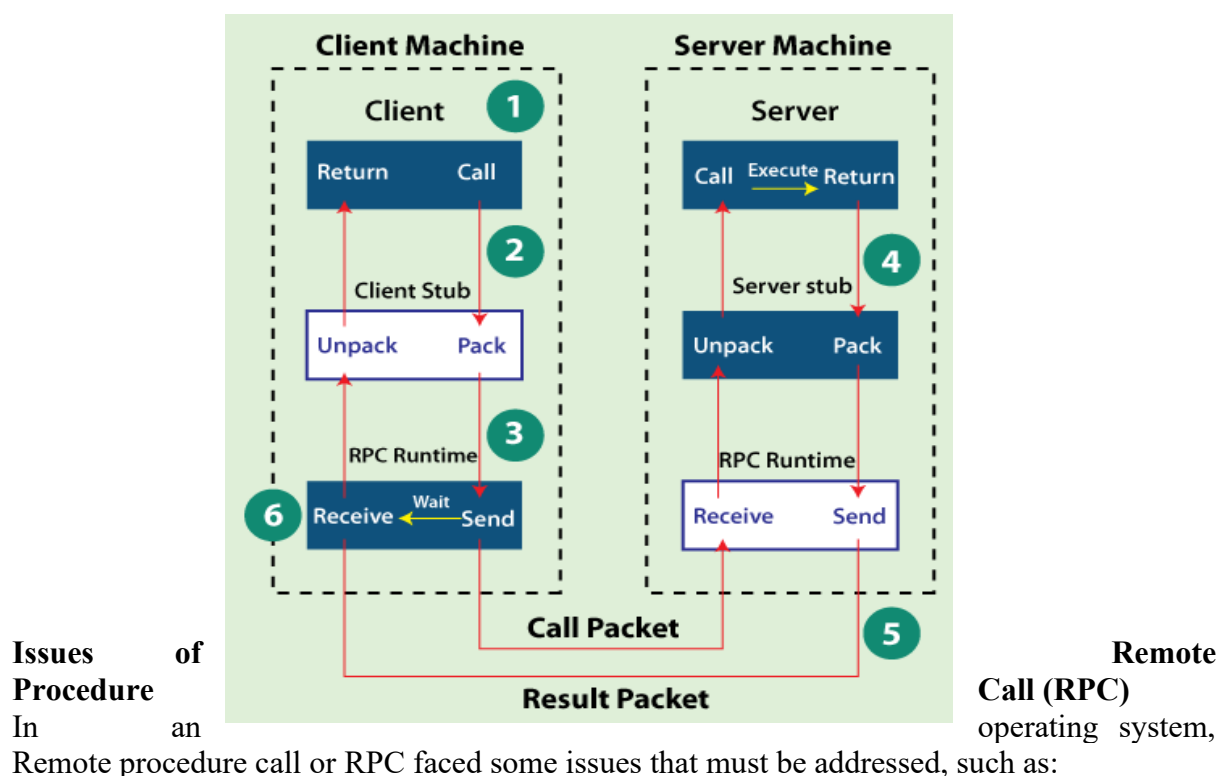
**Step 2:** A client starts a client stub process by passing parameters in the usual way. The packing of the procedure parameters is called **marshalling**. The client stub stores within the client's own address space, and it also asks the local RPC Runtime to send back to the server stub.

**Step 3:** In this stage, the user can access RPC by making regular Local Procedural Call. RPC Runtime manages the transmission of messages between the network across client and server, and it also performs the job of retransmission, acknowledgment, routing, and encryption.

**Step 4:** After completing the server procedure, it returns to the server stub, which packs (marshalls) the return values into a message. The server stub then sends a message back to the transport layer.

**Step 5:** In this step, the transport layer sends back the result message to the client transport layer, which returns back a message to the client stub.

**Step 6:** In this stage, the client stub demarshalls (unpack) the return parameters in the resulting packet, and the execution process returns to the caller.



## 1. RPC Runtime

The RPC runtime system is a library of routines and services that handle the network communications that underlie the RPC mechanism. In the course of an RPC call, client-side

and server-side runtime systems code handle binding, establish communications over an appropriate protocol, pass call data between the client and server, and handle communications errors.

## 2. Stub

The function of the stub is to provide transparency to the programmer-written application code.

- **On the client-side:** The stub handles the interface between the client's local procedure call and the runtime system, marshaling and unmarshaling data, invoking the RPC runtime protocol, and if requested, carrying out some of the binding steps.
- **On the server-side:** The stub provides a similar interface between the runtime system and the local manager procedures executed by the server.

## 3. Binding

How does the client know who to call and where the service resides?

The most flexible solution is to use dynamic binding and find the server at run time when the RPC is first made. The first time the client stub is invoked, it contacts a name server to determine the transport address at which the server resides. The binding consists of two parts:

- **Naming:** A Server having a service to offer exports an interface for it. Exporting an interface registers it with the system so that clients can use it.
- **Locating:** A Client must import an (exported) interface before communication can begin.

## 4. The calling semantics associated with RPC

It is mainly classified into the following choices,

- **Retry request message:** Whether to retry sending a request message when a server has failed, or the receiver didn't receive the message.
- **Duplicate filtering:** Remove the duplicate server requests.
- **Retransmission of results:** To resend lost messages without re-executing the operations at the server-side.

## Characteristics of RPC

Here are the essential characteristics of remote procedure call:

- The called procedure is in another process, which is likely to reside in another machine.
- The processes do not share address space.
- Parameters are passed only by values.
- RPC executes within the environment of the server process.
- It doesn't offer access to the calling procedure's environment.

## Code-

### Client

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define IP_PROTOCOL 0
#define PORT_NO 15050
#define NET_BUF_SIZE 32
#define cipherKey 'S'
#define sendrecvflag 0
#define nofile "File Not Found!"

void clearBuf(char * b) {
    int i;
    for (i = 0; i < NET_BUF_SIZE; i++)
        b[i] = '\0';
}

char Cipher(char ch) {
    return ch ^ cipherKey;
}

int sendFile(FILE * fp, char * buf, int s) {
    int i, len;
    if (fp == NULL) {
        strcpy(buf, nofile);
        len = strlen(nofile);
        buf[len] = EOF;
        for (i = 0; i <= len; i++)
            buf[i] = Cipher(buf[i]);
        return 1;
    }
    char ch, ch2;
    for (i = 0; i < s; i++) {
        ch = fgetc(fp);
        ch2 = Cipher(ch);
        buf[i] = ch2;
        if (ch == EOF)
            return 1;
    }
    return 0;
}

int main() {
    int sockfd, nBytes;
```

```

struct sockaddr_in addr_con;
int addrlen = sizeof(addr_con);

addr_con.sin_family = AF_INET;
addr_con.sin_port = htons(PORT_NO);
addr_con.sin_addr.s_addr = INADDR_ANY;
char net_buf[NET_BUF_SIZE];
FILE * fp;
sockfd = socket(AF_INET, SOCK_DGRAM, IP_PROTOCOL);
if (sockfd < 0)
    printf("\nfile descriptor not received!!\n");
else
    printf("\nfile descriptor %d received\n", sockfd);
if (bind(sockfd, (struct sockaddr * ) & addr_con, sizeof(addr_con)) == 0)
    printf("\nSuccessfully binded!\n");
else
    printf("\nBinding Failed!\n");
while (1) {
    printf("\nWaiting for file name...\n");
    clearBuf(net_buf);
    nBytes = recvfrom(sockfd, net_buf, NET_BUF_SIZE, sendrecvflag, (struct sockaddr * ) &
addr_con, &
    addrlen);
    fp = fopen(net_buf, "r");
    printf("\nFile Name Received: %s\n", net_buf);
    if (fp == NULL)
        printf("\nFile open failed!\n");
    else
        printf("\nFile Successfully opened!\n");
    while (1) {
        if (sendFile(fp, net_buf, NET_BUF_SIZE)) {
            sendto(sockfd, net_buf, NET_BUF_SIZE,
                sendrecvflag,
                (struct sockaddr * ) & addr_con, addrlen);
            break;
        }
        sendto(sockfd, net_buf, NET_BUF_SIZE,
            sendrecvflag,
            (struct sockaddr * ) & addr_con, addrlen);
        clearBuf(net_buf);
    }
    if (fp != NULL)
        fclose(fp);
}
return 0;
}

```

## Server

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define IP_PROTOCOL 0
#define PORT_NO 15050
#define NET_BUF_SIZE 32
#define cipherKey 'S'
#define sendrecvflag 0
#define nofile "File Not Found!"

void clearBuf(char * b) {
    int i;
    for (i = 0; i < NET_BUF_SIZE; i++)
        b[i] = '\0';
}

char Cipher(char ch) {
    return ch ^ cipherKey;
}

int sendFile(FILE * fp, char * buf, int s) {
    int i, len;
    if (fp == NULL) {
        strcpy(buf, nofile);
        len = strlen(nofile);
        buf[len] = EOF;
        for (i = 0; i <= len; i++)
            buf[i] = Cipher(buf[i]);
        return 1;
    }
    char ch, ch2;
    for (i = 0; i < s; i++) {
        ch = fgetc(fp);
        ch2 = Cipher(ch);
        buf[i] = ch2;
        if (ch == EOF)
            return 1;
    }
    return 0;
}

int main() {
    int sockfd, nBytes;
    struct sockaddr_in addr_con;
```



```

int addrlen = sizeof(addr_con);

addr_con.sin_family = AF_INET;
addr_con.sin_port = htons(PORT_NO);
addr_con.sin_addr.s_addr = INADDR_ANY;
char net_buf[NET_BUF_SIZE];
FILE * fp;
sockfd = socket(AF_INET, SOCK_DGRAM, IP_PROTOCOL);
if (sockfd < 0)
    printf("\nfile descriptor not received!!\n");
else
    printf("\nfile descriptor %d received\n", sockfd);
if (bind(sockfd, (struct sockaddr * ) & addr_con, sizeof(addr_con)) == 0)
    printf("\nSuccessfully binded!\n");
else
    printf("\nBinding Failed!\n");
while (1) {
    printf("\nWaiting for file name...\n");
    clearBuf(net_buf);
    nBytes = recvfrom(sockfd, net_buf, NET_BUF_SIZE, sendrecvflag, (struct sockaddr * ) &
addr_con, &
    addrlen);
    fp = fopen(net_buf, "r");
    printf("\nFile Name Received: %s\n", net_buf);
    if (fp == NULL)
        printf("\nFile open failed!\n");
    else
        printf("\nFile Successfully opened!\n");
    while (1) {
        if (sendFile(fp, net_buf, NET_BUF_SIZE)) {
            sendto(sockfd, net_buf, NET_BUF_SIZE,
                sendrecvflag,
                (struct sockaddr * ) & addr_con, addrlen);
            break;
        }
        sendto(sockfd, net_buf, NET_BUF_SIZE,
            sendrecvflag,
            (struct sockaddr * ) & addr_con, addrlen);
        clearBuf(net_buf);
    }
    if (fp != NULL)
        fclose(fp);
}
return 0;
}

```

### Output :

```
jarvis@J4RVIS:/mnt/d/Me/College/DSL/8_File$ ./test
Enter file name to request: Test.c
Receiving file...
```

### Client

```
jarvis@J4RVIS:/mnt/d/Me/College/DSL/8_File$ gcc Test.c -o test
jarvis@J4RVIS:/mnt/d/Me/College/DSL/8_File$ ./client

File descriptor 3 received

Successfully binded!

Waiting for file name...

File Name Received: Test.c

File Successfully opened!

Waiting for file name...
```

**Conclusion :** In this practical we learnt about RPC mechanism and its implementation for file transfer over network.

**Sign of Teacher**  
**Mrs. Archana Chitte**